Roteiro 8 — Configuração, Docker e pipeline

Disciplina: Desenvolvimento Web com .NET — UNIP. **Fonte:** Roteiros.pdf, páginas 24–26.
Consolidação: 05/11/2026, com preservação das evidências de outubro.

Objetivo, introdução e ferramentas

Separar código/configuração, empacotar API e frontend e impedir publicação de uma versão que falhe nas verificações. A implementação utiliza arquivos por ambiente, User Secrets/variáveis externas, Docker multi-stage, Compose, GitHub Actions e scripts PowerShell.

Configuração e ambientes

`appsettings.json` contém valores comuns e arquivos Development/Production contêm diferenças. Segredos ficam externos. `PortalOptions` valida o modo de armazenamento e a origem; a conexão é obrigatória para SQL Server. O modo Memory é restrito a Development/Testing e não representa uma alternativa de produção ao banco exigido.

O README lista variáveis, configuração local, seed e inicialização. A mesma fonte deve iniciar em desenvolvimento e produção simulada por configuração, sem alterar código. Esse cenário exige execução própria e não é comprovado apenas pela presença dos dois arquivos JSON.

Containerização e escolha de implantação

O Dockerfile usa estágios para frontend, backend e runtime, imagens base versionadas e usuário não privilegiado no processo final. O React é incorporado em `wwwroot/app`. O Compose fornece SQL Server, Web, portas locais, certificado externo, volume SQL e persistência das chaves Data Protection.

Estratégia	Consequência para o portal
React integrado à API	Um artefato operacional, mesma origem, cookies e roteamento mais simples
React em implantação separada	Releases independentes, exigindo configuração adicional de proxy/origem/sessão

A estratégia integrada foi selecionada pelo escopo acadêmico e pela presença simultânea de MVC/SPA. Nenhuma credencial deve ser copiada para a imagem; o PFX e o ambiente são montados externamente. A evidência obrigatória inclui build, execução do contêiner e Health Check aprovado.

CI, dependências e bloqueio de publicação

O workflow declara gatilhos de PR/branch principal e SDK/Node explícitos. A validação deve provisionar SQL Server dedicado, aguardar readiness, aplicar migrations no banco reservado e executar a suíte de integração. `Test-Integration.ps1` recusa banco fora do prefixo de teste e exige TRX com suítes obrigatórias, sem falhas/ignorados.

Frontend executa `npm ci`, auditoria de vulnerabilidades, tipos/contrato, testes e build. O cliente gerado precisa corresponder a OpenAPI real. Empacotamento/publicação ficam depois das verificações; uma falha deve encerrar o fluxo. O caminho aprovado já foi executado. A prova negativa ainda exige executar o workflow com falha controlada e verificar que nenhum artefato publicável foi produzido para aquele commit; um upload de diagnóstico de falha não equivale à publicação da aplicação.

A validação estática histórica conferiu parser YAML, sintaxe Bash/PowerShell, SQL pronto antes das migrations/testes e smoke antes de salvar imagem/publicar artefatos, sem `always` ou `continue-on-error` na publicação. Esse método isolado não inicia Docker nem GitHub Actions. A evidência remota abaixo acrescenta a execução real; a validação do workflow alterado nesta auditoria continua distinta do run anterior.

O ensaio negativo prepara falha intencional em teste simples, exclusivamente em branch experimental. O envio dessa branch e a execução no GitHub estão **pendentes de autorização explícita**. A branch principal não recebe o teste defeituoso. Depois do ensaio autorizado, deve-se registrar URL/SHA, a etapa reprovada, etapas de publicação `skipped` e ausência de artefato publicável, remover a falha e verificar a integridade da versão original. Nesta consolidação não se apresenta esse resultado como já executado.

O aprimoramento selecionado é o inventário reproduzível de dependências .NET/npm como artefato. Tag da imagem, versão informativa e manifesto de commit relacionam artefato ao código validado.

Os inventários .NET e npm locais foram preservados; a etapa equivalente de inventário do runner também foi aprovada. A relação da imagem construída ao commit foi confirmada nos logs de build e nos metadados do artefato.

Execução real aprovada no GitHub

O workflow 42117773881, `Validar e empacotar Portal de Tarefas`, foi consultado novamente pela API GitHub. Ele executou por `push` em `main`, commit **`1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`**, com estado `completed` e conclusão `success`. O intervalo registrado foi 05/11/2026, 22:00:55–22:03:18 UTC, equivalente a 19:00:55–19:03:18 de São Paulo. O job `validar`, ID `127561862888`, aprovou todas as etapas obrigatórias; somente as duas etapas condicionais de diagnóstico de falha foram ignoradas por não haver falha.

Verificação remota	Resultado confirmado
Restore/build .NET Release	Aprovado, zero avisos e zero erros
SQL Server, migrations e integração	Banco exclusivo; 55 aprovados, zero falhos e zero ignorados

Frontend	`npm ci`, auditoria, 33 testes aprovados em cinco arquivos, typecheck e build
Contrato	OpenAPI real, cliente gerado, comparação sem divergência e chamada TypeScript aprovados
Imagem	Build multi-stage integrado aprovado, tag com SHA completo
Execução	SQL Server e HTTPS; a etapa verificou que o UID do processo não era zero
Smoke	Liveness/readiness, `/version`, MVC, SPA e assets aprovados pelo script
Empacotamento	`docker save` e upload executados depois dos gates obrigatórios

Os logs identificam a imagem `portaltarefas:1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`, ID local Docker `sha256:c80953ffbbb3ff59232094808f251b33fd40375fc9d376e48130221944e1f80c`. A referência normalizada `docker.io/library/portaltarefas` na saída do build é o nome local do Docker e não comprova push ao Docker Hub ou GHCR.

O artefato 13456822922 chama-se `portaltarefas-1da2eead374d03fcfb62e5b0d7af17e2dbaf1cb2`, tem **159.303.801 bytes** e digest do ZIP **`sha256:650797a7b10c8b54271d8d4885626f40b091ef59c251a47a404007643e711e2e`**. A API informa expiração em 12/11/2026, 22:02:52 UTC. Essa retenção curta exige preservar os registros acadêmicos; a existência posterior do download não é garantida.

Foram preservados resumo e limites, etapas, metadados do artefato e trechos de resultados do log. O ZIP completo não foi extraído nesta consulta; os valores individuais de UID e do manifesto smoke não foram lidos de seu interior. A aprovação das etapas e o script executado confirmam as verificações e seus critérios de sucesso.

O smoke exige respostas HTTP 200 de live/ready e versão exata. O run aprovado não registrou explicitamente o estado nativo `.State.Health.Status` do Docker. Esta auditoria acrescenta ao CI a espera de `healthy` antes de salvar/publicar a imagem; a alteração é verificável localmente, mas seu resultado remoto depende de nova execução autorizada.

CD, promoção e rollback

`Publish-Local.ps1` prepara homologação com imagem local identificada, arquivo externo de ambiente e certificado. O script preserva a instância anterior e chama `Smoke-Test.ps1`, que verifica live/ready, versão, MVC e entrada da SPA. O procedimento não autoriza publicação remota automática.

Para um provedor futuro, conexões/credenciais devem vir de secret store ou identidade federada; parâmetros do destino não são inventados. A promoção reutiliza a imagem já testada, mantendo commit/tag. O rollback deve voltar à imagem anterior preservada e considerar separadamente compatibilidade das migrations e backup do banco. Volumes não devem ser apagados para reverter uma aplicação.

Na página 25, E4 exige imagem vinculada ao commit, identidade federada **ou** secret store do provedor, implantação em ambiente de teste com smoke/Health Check e estratégia de promoção/retorno com registro de versão. O PDF não exige GHCR, Docker Hub ou um provedor específico: a distribuição pode usar o artefato de imagem já validado. A implantação em staging com aprovação antes de produção aparece na página 26 como extensão alternativa. O inventário de dependências é a extensão selecionada, de modo que não se acrescenta infraestrutura de produção apenas por existir no projeto de referência.

O ambiente efêmero do runner implantou a imagem com SQL Server e HTTPS, verificou saúde, executou smoke e identificou a versão pelo SHA. Essa execução atende **E4.3, implantação em ambiente de teste**; a homologação local atual do mesmo artefato reforça a verificação. O PDF não determina que o ambiente seja permanente ou remoto. Não houve hospedagem persistente, promoção para produção ou rollback remoto; esses cenários não são apresentados como executados nem transformados em exigências adicionais. **E4.2, identidade federada ou secret store do provedor, continua pendente**, pois a configuração externa e as senhas efêmeras do CI não demonstram esse mecanismo. A estratégia de promoção/retorno e os scripts estão implementados; a promoção reutiliza exatamente a imagem validada, sem reconstruí-la.

Finalização técnica sem novas capturas

Nesta rodada foram preservados os sete relatórios anteriores e todas as imagens existentes. O plano visual final é apenas um checklist para execução posterior. Não foram instaladas ferramentas de captura nem executadas automações de navegador para screenshots nesta etapa.

O diagnóstico Docker Windows e seu registro de comandos de 29/10 comprovam Docker CLI 29.9.2 e Compose 5.6.1 portáteis. A configuração Compose rejeitou variáveis obrigatórias ausentes e aceitou marcadores sintéticos definidos somente no processo. `docker info` retornou código 1 por ausência de Engine/WSL. A reprodução local continua dependendo desses pré-requisitos; o build, a execução, o usuário não privilegiado e o smoke no runner já foram comprovados pela execução de 05/11. Nenhuma instalação global foi feita para resolver o bloqueio Windows.

A configuração de homologação separa o projeto `portal-tarefas-tests` e exige banco reservado de teste. O namespace anterior pode ser mantido explicitamente por `PORTAL_COMPOSE_PROJECT=portal-tarefas`, preservando seus volumes. O script de publicação confere SHA completo da imagem contra a revisão OCI, valida o banco, aplica migrations apenas mediante opção explícita, compartilha chaves quando configurado e registra manifesto sanitizado e comandos de rollback. As guardas e sua execução negativa são verificações próprias; não substituem uma publicação Docker real.

Resultados consolidados e limites

A regressão local de 05/11/2026 aprovou **55 testes .NET com SQL real e 33 testes React**, zero falhos, zero ignorados e zero bloqueados nessas duas suítes. Restore/build .NET Release, typecheck e build frontend passaram. O resumo SQL e os resultados frontend identificam os comandos e a execução. O banco exclusivo `PortalTarefas_Tests_Final20261105_26b194a7` não foi compartilhado com seed/homologação.

Em outro banco novo reservado, o seed repetido manteve três tarefas, três auditorias, duas roles, dois usuários sintéticos e três migrations. O contrato HTTPS e os tipos gerados permaneceram idênticos aos versionados. Cliente de leitura e cliente autenticado passaram com validação TLS. A publicação em pasta nos dois ambientes retornou HTTP 200 em live/ready, MVC e SPA, versão exata do artefato de origem e documentação restrita em Production. Processos de teste próprios foram encerrados; dados e volumes preservados.

O ensaio local isolado produziu uma reprovação intencional, sem ignorados, e a reversão restaurou 29 testes de domínio/memória aprovados. Esses 29 são um subconjunto da suíte normal, e a falha esperada não é uma regressão do produto. A branch experimental possui commits reais de preparação, falha e reversão descritos no procedimento. Sua execução remota continua pendente de autorização.

A validação estática do CI atualizado aprovou 31 etapas YAML, 22 blocos Bash, seis scripts e quatro blocos PowerShell. O teste local do novo gate de saúde aprovou três cenários com funções Docker/sleep simuladas: aceita `healthy`, rejeita `unhealthy` e timeout. Essa checagem da lógica não é evidência de Docker real. A consulta Docker no Windows reconfirmou a indisponibilidade do Engine, preservando a aprovação remota anterior.

Os resultados e bloqueios estão consolidados no status de execução. O CI remoto, o build/execução da imagem e a implantação efêmera de teste com smoke HTTP estão aprovados para o commit identificado. Permanecem distintos: reprodução Docker no Windows sem Engine, nova execução do gate de saúde nativo, ensaio de falha controlada pendente de autorização e mecanismo de identidade/segredos do provedor ainda não demonstrado.

O roteiro 8 permanece **parcial** por V3, prova negativa remota, e E4.2, identidade federada/secret store do provedor. Não foram feitas novas capturas; as imagens e os relatórios 1–7 permanecem preservados. O PDF 08 é gerado a partir deste Markdown pelo processo existente, com hash verificado e sem produzir screenshots.

Como histórico de 29/10/2026, os smokes Development e Production simulada foram executados sobre o mesmo binário publicado: live, ready, MVC e SPA retornaram 200 e `/version` informou o commit exato `47c5f2fe37768d16c1a009b9b15688941e04bb02`. Development recebeu User Secrets em perfil isolado e Production recebeu variáveis externas. O manifesto do artefato local histórico identifica DLL/assets e seus SHA256. Essa execução antiga comprova publicação local e configuração por ambiente; não é apresentada como execução Docker, workflow remoto ou CD por imagem. Os novos smokes de 05/11, vinculados ao SHA de origem `1da2eea`, estão identificados separadamente acima.